

PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

ServiceNow Micro Project | SmartBridge

Item	Details
Team ID	SWTID-2026-3598
Team Size	5
Team Leader	VEERAMANI B
Team Members	BALAJI E, Anbumani A, NAGULAN.C, Kathir
Platform	ServiceNow (Incident Management)
Key Skills	Incident Management, UI Policy, UI Policy Actions, Client Scripts, Form Validation

1. Brainstorming & Ideation Phase

1.1 Problem Statement

Incident records need consistent and accurate data entry so that triage, routing and resolution work effectively. Relying only on user awareness and manual checks leads to incomplete, inconsistent or incorrect data. This hurts reporting accuracy, SLA compliance and overall service quality. Field behavior and validation therefore need to be enforced directly at the user-interface level.

1.2 Objective

To demonstrate how ServiceNow client-side controls (UI Policies and Client Scripts) enforce data integrity on Incident records by:

- making fields mandatory or read-only based on conditions;
- auto-populating dependent field values;
- blocking record submission when business rules are not met;
- preventing uncontrolled edits from the list view.

1.3 Ideas Considered

Idea	Description	Decision
Conditional mandatory field	Make Assignment group mandatory when Impact is High.	Selected (UI Policy)
Read-only dependent field	Lock Urgency when Impact is High so it cannot be changed manually.	Selected (UI Policy Action)
Auto-set urgency	Automatically set Urgency to High when Impact changes to High.	Selected (onChange script)
Save-time validation	Block save if Assigned To is empty on a High impact incident.	Selected (onSubmit script)
List-edit protection	Stop users changing State directly from the list.	Selected (onCellEdit script)
Server-side Business Rule	Enforce the same rules on the server.	Out of scope (client-side focus)

1.4 Proposed Solution

A combination of one UI Policy (with two actions) and three Client Scripts on the Incident table. The UI Policy handles declarative behavior (mandatory / read-only with Reverse if false), while the Client Scripts handle logic that a UI Policy cannot: auto-setting values, cancelling a save, and blocking list edits.

2. Requirement Analysis Phase

2.1 Functional Requirements

ID	Requirement	Implemented By
FR-1	When Impact = 1 - High, Assignment group must be mandatory.	UI Policy: High Impact Control
FR-2	When Impact = 1 - High, Urgency must be read-only.	UI Policy Action (Urgency)
FR-3	When Impact is no longer High, the mandatory/read-only behavior must be reversed.	Reverse if false = true
FR-4	When Impact changes to High, Urgency is automatically set to 1 - High and an info message is shown.	onChange Client Script
FR-5	An Incident with Impact = High cannot be saved if Assigned To is empty; an error is shown on the field.	onSubmit Client Script
FR-6	State cannot be changed by editing a cell in the list view; an alert is shown and the value stays unchanged.	onCellEdit Client Script
FR-7	State can still be changed from the Incident form.	Default form behavior

2.2 Non-Functional Requirements

- Lightweight: client-side logic only, no extra tables or integrations.
- Usability: clear, specific messages so users know how to fix the data.
- Maintainability: descriptive script names and small, readable scripts.
- Reversibility: policies revert automatically when conditions no longer apply.

2.3 Technical Requirements

Requirement	Detail
Platform	ServiceNow instance (Personal Developer Instance)
Access	Administrative or configuration access (e.g. admin role)
Table	Incident [incident]
Navigation	System UI > UI Policies; System UI > Client Scripts; Incident > Create New / Open / All
Scope	Global application scope
Language	JavaScript (ServiceNow client-side APIs: g_form, callback)

2.4 User Stories

- **US-1:** As a service desk analyst, I want Assignment group and Assigned To enforced on high impact incidents so that critical issues are never left unrouted.

- **US-2:** As a manager, I want Urgency locked on high impact incidents so that priority stays consistent for reporting and SLAs.
- **US-3:** As an administrator, I want State protected from list edits so that lifecycle changes go through the full form.

2.5 Assumptions & Constraints

- Client-side rules can be bypassed by API or imports; server-side rules would be needed for full enforcement.
- Scripts run on the Desktop UI type as configured.

3. Project Design Phase

3.1 Solution Architecture

All components sit on the Incident table and execute in the browser when the form or list is used:

Component	Trigger	Responsibility
UI Policy: High Impact Control	Form load and Impact change	Applies field actions when Impact = 1 - High; reverses them otherwise.
UI Policy Action: Assignment group	Policy condition true	Mandatory = True, Read-only = False.
UI Policy Action: Urgency	Policy condition true	Read-only = True, others left alone.
Client Script: onChange (Impact)	Impact value changes	Sets Urgency to 1 and shows an info message.
Client Script: onSubmit	User submits/saves form	Blocks save if Impact is High and Assigned To is empty.
Client Script: onCellEdit (State)	Inline edit in list view	Shows alert and cancels the edit.

3.2 Process Flow

1. User opens a new or existing Incident form.
2. User sets Impact to 1 - High.
3. UI Policy makes Assignment group mandatory and Urgency read-only.
4. onChange script sets Urgency to 1 - High and displays an information message.
5. User clicks Submit / Update. The onSubmit script checks Assigned To.
6. If Assigned To is empty, an error is shown on the field and the save is cancelled; otherwise the record is saved.
7. If Impact is later changed away from High, Reverse if false removes the mandatory and read-only behavior.

3.3 Design Decisions

Decision	Rationale
UI Policy for mandatory / read-only	Declarative, no code, and supports Reverse if false automatically.
Client Script for auto-set Urgency	UI Policies cannot conditionally set values with messages; onChange can.
onSubmit for Assigned To	Provides save-time validation and a field-level error message.
onChange guard: isLoading newValue == ""	Prevents the script from firing when the form first loads or the field is cleared.
onCellEdit callback(false)	Cancels the inline change so the State value stays as it was.

4. Project Planning Phase

4.1 Work Breakdown

Task	Activity	Deliverable	Status
Task 1	Create UI Policy "High Impact Control" with condition Impact is 1 - High and Assignment group action	Active UI Policy	Completed
Task 2	Create UI Policy Action for Urgency (Read-only)	Urgency action	Completed
Task 3	Create onChange Client Script "Auto set urgency for high impact"	Client Script	Completed
Task 4	Create onSubmit Client Script "Prevent save if Assigned To missing"	Client Script	Completed
Task 5	Create onCellEdit Client Script "Prevent state change via list edit"	Client Script	Completed
Task 6	Test configuration (mandatory, save, reverse, list edit, form update)	Test evidence	Completed

4.2 Team Structure

Role	Team Member(s)
Team Leader	VEERAMANI B
Team Members	BALAJI E, Anbumani A, NAGULAN.C, Kathir

Suggested split of work: configuration of UI Policy and actions (Tasks 1-2), client scripting (Tasks 3-5), testing and screenshots (Task 6), and documentation and demo preparation. Update the responsibilities to match your team's actual allocation.

4.3 Risks & Mitigation

Risk	Mitigation
Script fires while the form is loading	Use the isLoading guard in onChange.
Conflicting behavior between UI Policy and scripts	Keep responsibilities separate: UI Policy for mandatory/read-only, scripts for values and validation.
Wrong field names in scripts	Use back-end names (impact, urgency, assigned_to).
Client-side rules bypassed via API	Document as a limitation; recommend Business Rules as a future enhancement.

5. Project Development Phase

5.1 Task 1 - UI Policy on Incident

Navigate to System UI > UI Policies and click New.

Field	Value
Short description / Name	High Impact Control
Table	Incident [incident]
Active	true
Condition	Impact is 1 - High
Reverse if false	true
On load	true
Global	true

UI Policy Action - Assignment group:

Setting	Value
Field name	Assignment group
Mandatory	True
Visible	Leave alone
Read only	False

5.2 Task 2 - UI Policy Action: Urgency

From the High Impact Control policy, open the UI Policy Actions related list and click New.

Setting	Value
Field name	Urgency
Mandatory	Leave alone
Visible	Leave alone
Read only	True

5.3 Task 3 - onChange Client Script

Field	Value
Name	Auto set urgency for high impact
Table	Incident [incident]
UI Type	Desktop

Field	Value
Type	onChange
Field name	Impact
Active	true

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

5.4 Task 4 - onSubmit Client Script (Save Validation)

Field	Value
Name	Prevent save if Assigned To missing
Table	Incident [incident]
UI Type	Desktop
Type	onSubmit
Active	true

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

5.5 Task 5 - onCellEdit Client Script

Field	Value
Name	Prevent state change via list edit
Table	Incident [incident]
UI Type	Desktop
Type	onCellEdit

Field	Value
Field name	State
Active	true

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);  
}
```

5.6 Development Notes

- Scripts run in strict mode with isolated scope, so direct DOM, jQuery and window access are disabled; only `g_form` and supported APIs are used.
- Assignment group is enforced by the UI Policy, while Assigned To is enforced at save time by the `onSubmit` script.
- Priority updates automatically from Impact and Urgency (High/High gives Priority 1 - Critical) through standard Incident behavior.

6. Project Testing Phase

6.1 Test Approach

Manual functional testing on the Incident form and list view, covering the positive path, the blocking path and the reverse condition.

6.2 Test Cases

ID	Scenario	Steps	Expected Result	Result
TC-1	Mandatory enforcement	Incident > Create New; set Impact = High; leave Assigned To empty; click Submit.	Save blocked; error "Assigned To is mandatory for High impact incidents." shown on the field.	Pass
TC-2	Urgency auto-set	Set Impact = High on the form.	Urgency becomes 1 - High and is read-only; info message displayed.	Pass
TC-3	Assignment group mandatory	Set Impact = High.	Assignment group is marked mandatory.	Pass
TC-4	Successful save	Fill Assigned To (e.g. Beth Anglin); click Submit.	Incident saved without errors.	Pass
TC-5	Reverse condition	Open a High impact incident; change Impact to Medium.	Assigned To no longer mandatory; Urgency editable; record saves.	Pass
TC-6	List edit blocking	Incident > All; double-click State in a row and change it.	Alert "State cannot be changed from the list..." shown; State unchanged.	Pass
TC-7	Form-based update	Open the incident; change State to In Progress; click Update.	State change saved successfully.	Pass

6.3 Observed Results

- Saving a High impact incident without Assigned To was blocked with an inline error.
- After choosing an assignee, the incident saved as INC0010012 with Urgency 1 - High and Priority 1 - Critical.
- Changing Impact to 2 - Medium relaxed the policy behavior and allowed a normal save.
- Inline State edit in the list raised the alert and kept the original value, while editing State on the form worked as expected.

6.4 Defects / Observations

No blocking defects were found. Observation: because the rules are client-side, they do not apply to records created through APIs or import sets.

7. Project Documentation Phase

7.1 Documentation Deliverables

Artifact	Description
Project step-by-step guide	Reference PDF describing Tasks 1-6 with screenshots.
Configuration summary	Table of UI Policy, UI Policy Actions and Client Scripts (Section 7.2).
Scripts	Source of the onChange, onSubmit and onCellEdit scripts (Section 5).
Test evidence	Test cases and screenshots from Task 6 (Section 6).
This document	Consolidated project documentation across all eight phases.

7.2 Configuration Summary

Name	Type	Applies To	Purpose
High Impact Control	UI Policy	Incident	Condition: Impact is 1 - High; Reverse if false.
Assignment group action	UI Policy Action	Assignment group	Mandatory = True.
Urgency action	UI Policy Action	Urgency	Read only = True.
Auto set urgency for high impact	onChange	Impact	Set Urgency = 1 and show info message.
Prevent save if Assigned To missing	onSubmit	Incident form	Block save when High impact and no assignee.
Prevent state change via list edit	onCellEdit	State (list)	Alert and cancel inline edit.

7.3 Maintenance Guidelines

- To disable a rule temporarily, set Active = false on the specific UI Policy or Client Script.
- If Impact/Urgency choice values change, update the hard-coded values ('1') in the scripts.
- Retest all scenarios in Section 6 after any change.

7.4 Limitations & Future Enhancements

- Add server-side Business Rules or Data Policies so rules also apply to API and import updates.
- Extend the pattern to other fields (for example Category or Short description) and other tables such as Problem or Change.
- Use UI Policy scripts or Catalog-style messages for richer user guidance.

7.5 Conclusion

The project shows that UI Policies and Client Scripts work together to enforce dynamic field behavior, automate updates and prevent incorrect data submission on Incident forms. Fields such as Assignment group, Assigned To and Urgency respond correctly to changes in Impact, keeping data clean and

consistent. The solution is lightweight, easy to implement and follows good practice for form usability and data integrity in ServiceNow.

8. Project Demonstration Phase

8.1 Demo Objective

Show live that the configured UI Policy and Client Scripts enforce the business rules on Incident records, and that the behavior reverses when the condition no longer applies.

8.2 Pre-Demo Checklist

- Logged in to the ServiceNow instance with admin access.
- UI Policy High Impact Control and all three Client Scripts are Active.
- At least one existing incident available in the list for the list-edit demo.

8.3 Demo Script

Step	Action	What to Point Out
1	Show System UI > UI Policies and open High Impact Control.	Condition Impact is 1 - High, Reverse if false, and the two actions.
2	Show the three Client Scripts under System UI > Client Scripts.	onChange (Impact), onSubmit, onCellEdit (State).
3	Incident > Create New; set Impact = High.	Urgency auto-set to High and locked; Assignment group mandatory; info message.
4	Click Submit without Assigned To.	Save blocked with inline error on Assigned To.
5	Select an assignee and Submit.	Record saves successfully.
6	Open the incident and change Impact to Medium.	Mandatory/read-only reversed; Urgency editable.
7	Incident > All; double-click State in the list.	Alert appears and State stays unchanged.
8	Open the incident form and change State, then Update.	Form-based update works as expected.

8.4 Key Talking Points

- Difference between a UI Policy (declarative) and a Client Script (programmatic), and why both are used.
- Purpose of Reverse if false and of the isLoading guard in onChange.
- Benefits: cleaner data, better SLA reporting, fewer manual checks.
- Limitations and next steps (server-side enforcement).

8.5 Expected Outcome

By the end of the demo, the audience should see that High impact incidents are consistently routed with an assignment group and assignee, urgency is controlled, and State is protected from unintended list edits, all without any changes to back-end logic.